



UMAD[®]

UNIVERSIDAD MADERO
MÉXICO

Primer Parcial

Adolfo Salinas Bautista

20122784

Ingenieria de software

Plataformas propietarias I

Prof.Nicolas Arrioja Landa Cosio

Introducción

El presente proyecto muestra una propuesta de mejoras respecto a métodos básicos dentro del paradigma de la programación orientado a objetos. Este sistema implementa conceptos del uso de interfaces nativas del framework .NET como IEnumerable, IComparable, etc.

Antecedentes

Tradicionalmente, en la enseñanza inicial de la programación, la gestión de colecciones se realiza mediante iteraciones simples (ciclos for o foreach) sobre arreglos o listas, donde cualquier lógica de filtrado (como ignorar elementos inactivos) o de ordenamiento debe ser programada explícitamente en el bloque principal de ejecución.

Este enfoque, aunque funcional para tareas triviales, conlleva problemas de escalabilidad y mantenimiento:

- Código repetitivo: La lógica de filtrado se repite cada vez que se recorre la lista.
- Acoplamiento: El programa principal debe conocer los detalles internos de cómo se almacenan y filtran los datos.
- Desorden: Sin una implementación robusta de comparación, los datos carecen de una jerarquía clara de importancia.

Descripción del problema

Los agentes de soporte técnico a menudo se enfrentan a listas de tareas desorganizadas donde se mezclan incidencias críticas con tareas triviales, y tickets ya resueltos con tickets pendientes. Esto genera ineficiencia operativa y riesgo de ignorar problemas urgentes.

Objetivo General

Desarrollar un sistema encargado de la gestión de casos para el módulo de soporte técnico en C# para automatizar mediante nivel de importancia.

Objetivos Específicos

- Definir un contrato de comportamiento mediante la interfaz ITicket que estandarice las propiedades y métodos esenciales de cualquier solicitud de soporte.

- Implementar lógica de comparación personalizada utilizando IComparable en la entidad SoporteBoleto para establecer una jerarquía descendente basada en la prioridad (Alta > Media > Baja).
- Desarrollar un iterador personalizado (IEnumerator) que encapsule la lógica de recorrido, permitiendo saltar automáticamente los tickets cuyo estado sea "Cerrado" durante la iteración.
- Construir una colección contenedora (IEnumerable) que integre el iterador personalizado y exponga los datos de forma segura al programa principal.
- Aplicar el Principio de Responsabilidad Única (SRP) mediante la separación del código en archivos distintos para enums, interfaces, lógica de negocio y ejecución.

Alcances

- Arquitectura: Aplicación de consola estructurada en múltiples archivos (.cs) bajo un mismo espacio de nombres.
- Gestión de Prioridades: Uso de Enums para definir niveles de severidad y algoritmos de comparación para el ordenamiento automático.
- Filtrado Inteligente: Capacidad de ocultar tickets cerrados de la vista principal sin eliminarlos de la memoria, únicamente mediante la lógica del iterador MoveNext().
- Interacción: Simulación de creación, atención (cierre) y listado de tickets.
- Tecnología: Uso exclusivo de C# y el framework .NET estándar, sin dependencias de terceros.

Limitaciones

- Persistencia de Datos: El sistema opera en memoria (RAM). Todos los tickets creados se pierden al cerrar la ejecución de la consola, ya que no se conecta a una base de datos.
- Tamaño Estático: Se utiliza una estructura de arreglo (Array) simple para la colección, lo que implica un límite fijo de tickets que se pueden almacenar, a diferencia de listas dinámicas.
- Interfaz de Usuario: La interacción se limita a la línea de comandos (CLI), careciendo de una interfaz gráfica (GUI) o web.
- Concurrencia: El sistema está diseñado para un solo usuario (monohilo) y no soporta acceso simultáneo de múltiples agentes.

Bibliografía

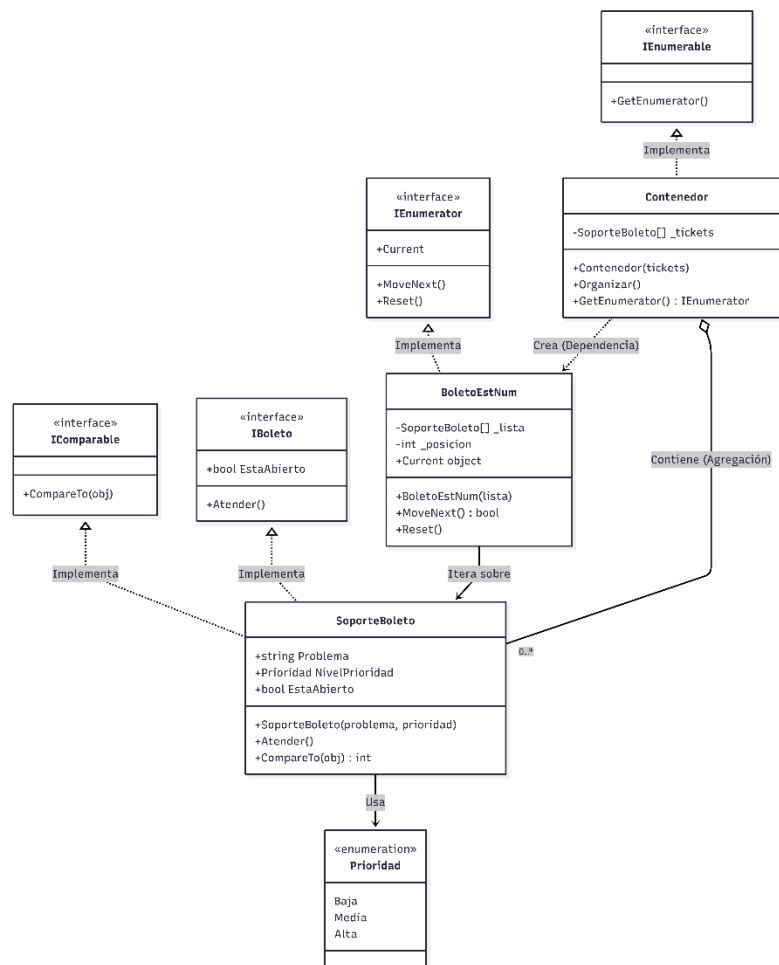
nicosiored. (2017, 15 de enero). *Tutorial C# nivel intermedio 2 --- Interfaces* [Video]. YouTube. <https://www.youtube.com/watch?v=KbveVrFqgK0>

nicosiored. (2017, 15 de enero). *Tutorial C# nivel intermedio 3 --- IEnumerable e IEnumerator parte I* [Video]. YouTube. <https://www.youtube.com/watch?v=bv9CqGjnSI0>

nicosiored. (2017, 15 de enero). *Tutorial C# nivel intermedio 4 --- IEnumerable e IEnumerator parte II* [Video]. YouTube. <https://www.youtube.com/watch?v=uKGea9sRVJ0>

nicosiored. (2017, 27 de enero). *Tutorial C# nivel intermedio 6 --- IComparable* [Video]. YouTube. <https://www.youtube.com/watch?v=gQH0mak38WE>

Diagrama



Código

```
using System;

namespace primerParcial
{
    /// <summary>
    /// Define los niveles de prioridad para los boletos de soporte.
    /// </summary>
    /// Autor: Adolfo Salinas
    /// Fecha: 08/02/2026
    public enum Prioridad
    {
        Baja = 1,
        Media = 2,
        Alta = 3
    }
}

using System;

namespace primerParcial
{
    /// <summary>
    /// Define la interfaz para los comportamientos básicos de un boleto de
    soporte.
    /// </summary>
    /// Autor: Adolfo Salinas
    /// Fecha: 08/02/2026
    public interface IBoleto
    {
        /// <summary>
        /// Realiza la atención del boleto, cambiando su estado.
    }
```

```

    /// </summary>
    void Atender();

    /// <summary>
    /// Indica si el boleto se encuentra activo o cerrado.
    /// </summary>
    bool estadoBoleto { get; }
}

}

using System;

namespace primerParcial
{
    /// <summary>
    /// Representa un ticket de soporte concreto. Implementa IBoleto y IComparable
    para ordenamiento.
    /// </summary>
    /// Autor: Adolfo Salinas
    /// Fecha: 08/02/2026
    public class SoporteBoleto : IBoleto, IComparable
    {
        public string Problema { get; set; }
        public Prioridad NivelPrioridad { get; set; }
        public bool estadoBoleto { get; set; }

        /// <summary>
        /// Constructor del boleto de soporte.
        /// </summary>
        /// Autor: Adolfo Salinas
        /// Fecha: 08/02/2026
        /// <param name="problema">Descripción del problema.</param>

```

```

    /// <param name="p">Nivel de prioridad.</param>
    public SoporteBoleto(string problema, Prioridad p)
    {
        Problema = problema;
        NivelPrioridad = p;

        // Por defecto el boleto podría inicializarse en true, dependiendo de
la lógica de negocio
        estadoBoleto = true;
    }
    /// <see cref="IBoleto.Atender()"/>
    public void Atender()
    {
        Console.WriteLine("--Operando--");
        estadoBoleto = false;
    }
    /// <summary>
    /// Compara este boleto con otro para determinar su orden basado en la
prioridad.
    /// </summary>
    /// <param name="obj">Objeto contra el cual comparar.</param>
    /// <returns>Valor entero que indica la posición relativa (orden
descendente).</returns>
    public int CompareTo(object obj)
    {
        SoporteBoleto sigBol = obj as SoporteBoleto;
        if (sigBol == null) return 1;

        // Documentamos la parte importante: Ordenamiento descendente por
prioridad

        return sigBol.NivelPrioridad.CompareTo(this.NivelPrioridad);
    }
}
}

```

```

using System;

using System.Collections;

namespace primerParcial
{
    /// <summary>
    /// Implementación concreta del Iterador. Recorre la lista de boletos saltando
    los inactivos.
    /// </summary>
    /// Autor: Adolfo Salinas
    /// Fecha: 08/02/2026
    public class BoletoEstNum : IEnumerator
    {
        private SoporteBoleto[] _lista; //!< La colección de boletos a recorrer.
        private int _posicion = -1; //!< Posición actual del iterador.

        /// <summary>
        /// Constructor. Recibe la colección a iterar.
        /// </summary>
        /// Autor: Adolfo Salinas
        /// Fecha: 08/02/2026
        /// <param name="lista">Arreglo de boletos de soporte.</param>
        public BoletoEstNum(SoporteBoleto[] lista)
        {
            _lista = lista;
        }

        /// <see cref="IEnumerator.MoveNext()"/>
        public bool MoveNext()
        {
            _posicion++;

```



```

        // Documentamos la lógica de filtrado: Salta los boletos que ya no
        tienen estado activo (false)

        while (_posicion < _lista.Length && !_lista[_posicion].estadoBoleto)
        {
            _posicion++;
        }

        return (_posicion < _lista.Length);
    }

    /// <see cref="IEnumerator.Reset()"/>
    public void Reset()
    {
        _posicion = -1;
    }

    /// <see cref="IEnumerator.Current"/>
    public object Current
    {
        get
        {
            {
                if (_posicion < 0 || _posicion >= _lista.Length)
                    throw new InvalidOperationException();

                return _lista[_posicion];
            }
        }
    }
}

using System;
using System.Collections;

namespace primerParcial
{
    /// <summary>

```

```
    /// Clase contenedora que almacena los boletos y provee el iterador
    personalizado.
```

```
    /// </summary>
```

```
    /// Autor: Adolfo Salinas
```

```
    /// Fecha: 08/02/2026
```

```
    public class Contenedor : IEnumerable
```

```
    {
```

```
        private SoporteBoleto[] _boletos;    ///!< Estructura interna de
    almacenamiento.
```

```
        /// <summary>
```

```
        /// Constructor del contenedor.
```

```
        /// </summary>
```

```
        /// Autor: Adolfo Salinas
```

```
        /// Fecha: 08/02/2026
```

```
        public Contenedor(SoporteBoleto[] boletos)
```

```
        {
```

```
            _boletos = boletos;
```

```
        }
```

```
        /// <summary>
```

```
        /// Ordena los boletos internos utilizando la implementación de
    IComparable.
```

```
        /// </summary>
```

```
        public void Organizar()
```

```
        {
```

```
            Array.Sort(_boletos);
```

```
        }
```

```
        /// <summary>
```

```
        /// Obtiene el enumerador personalizado para recorrer los boletos activos.
```

```

        /// </summary>
        /// <returns>Instancia de BoletoEstNum.</returns>
        public IEnumerator GetEnumerator()
        {
            return new BoletoEstNum(_boletos);
        }
    }
}

using System;

namespace primerParcial
{
    /// <summary>
    /// Clase principal que ejecuta la simulación del sistema de tickets.
    /// </summary>
    /// Autor: Adolfo Salinas
    /// Fecha: 08/02/2026
    class Program
    {
        /// <summary>
        /// Punto de entrada principal de la aplicación de consola.
        /// </summary>
        /// <param name="args">Argumentos de línea de comandos.</param>
        static void Main(string[] args)
        {
            // Creación de objetos simulando diferentes prioridades
            SoporteBoleto t1 = new SoporteBoleto("Pantalla azul",
            Prioridad.Alta); // Prioridad 3

            SoporteBoleto t2 = new SoporteBoleto("Mouse sucio", Prioridad.Baja);
            // Prioridad 1

```

```

        SoporteBoleto t3 = new SoporteBoleto("Servidor caído",
Prioridad.Alta); // Prioridad 3

        SoporteBoleto t4 = new SoporteBoleto("Actualizar Software",
Prioridad.Media); // Prioridad 2

// Simulamos la atención de un ticket antes de agregarlo al contenedor
final

        t3.Atender();

// Inicializamos el contenedor (Aggregate)
Contenedor bandeja = new Contenedor(new SoporteBoleto[] { t1, t2, t3,
t4 });

Console.WriteLine("\n--- Organizando por Prioridad ---");
// Se utiliza IComparable implícito en Array.Sort dentro de Organizar
bandeja.Organizar();

Console.WriteLine("--- Tickets Pendientes ---");

// Recorrido usando el Patrón Iterator (BoletoEstNum) implícito en el
foreach

        foreach (SoporteBoleto sp in bandeja)
        {
            Console.WriteLine($"[PENDIENTE] Prioridad: {sp.NivelPrioridad} |
{sp.Problema}");
        }
        Console.ReadLine();
    }
}
}

```